Formal Validation Enables Reliable AI-Generated Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 17, 2026

Abstract. A seismologist needs to image Earth's subsurface; a combustion engineer must predict ignition timing; an epidemiologist wants to forecast disease spread. Each builds a bespoke numerical solver from scratch—correct for one problem, unusable for the next. Large language models can now generate such solvers from plain-English descriptions, but the generated code silently fails on most non-textbook problems. Here we introduce a Judge Agent that applies well-posedness proofs, stability analysis, and domain-specific quality audits between AI planning and execution. Without the Judge, AI-generated code fails on 7 of 12 frontier problems; with it, all 12 produce correct solutions with automated error bounds. We validate on four domains with real data—clinical CT imaging, seismic inversion, combustion chemistry, and COVID-19 dynamics—and confirm generalization on 72 held-out tasks from 12 external scientists. Code, data, and benchmark are archived on Zenodo.

Introduction

In 2024, a structural engineer in Tokyo spent three weeks coding a finite-element solver for a novel bridge design. The code ran, the numbers looked reasonable, and the project moved forward—until a post-mortem revealed that an incorrect boundary condition had silently produced unconservative stress estimates throughout. This pattern repeats across science: thousands of researchers independently build bespoke numerical solvers—finite-element codes for bridges, quantum-chemistry packages for molecules, climate models for the atmosphere—each correct for one domain but unusable for any other [9]. Large language models (LLMs) can now generate such code from natural-language descriptions [7, 8, 20], much as AlphaFold [6] democratized protein structure prediction. But the generated code comes with no guarantee of correctness. When it silently produces a wrong answer—a laminar solution for a turbulent flow [18], a missing correlation term in a quantum calculation, a numerically unstable integrator for a stiff system—the scientist has no way to detect the error.

The mathematical foundations for automated error bounds exist: Lax–Richtmyer convergence theory [2] for discretizations, Hadamard well-posedness [3] for continuous dependence, and computability theory [1, 4] for fundamental limits. What has been missing is a practical system that *automatically* applies these classical results to arbitrary problem descriptions—and evidence that this works on problems scientists actually struggle with. Existing approaches address parts of this challenge: physics-informed neural networks (PINNs) [7, 21] learn solutions but lack automated error bounds; automated code generation from variational forms (FEniCS [22], Firedrake [23]) automates solver construction but requires expert problem formulation; formal verification tools [24] guarantee floating-point correctness but do not automate problem specification; and recent LLM-based scientific agents [25, 26] generate experimental protocols but not numerically certified simulations.

We present a three-agent pipeline—Plan ($\mathcal{A}_P$), Judge ($\mathcal{A}_J$), Execute ($\mathcal{A}_E$)—that translates natural-language problem descriptions into bounded-error numerical solutions (Fig. 1). The key

contribution is the Judge Agent, which operates in two modes: *pre-execution*, it validates well-posedness, dimensional consistency, and error budgets; *post-execution*, it audits solution quality against domain-specific physical constraints (conservation, monotonicity, entropy, symmetry). We emphasize that the underlying mathematics is entirely classical (Remark 3); the contribution is an engineering architecture that automates the application of these results to arbitrary problems from natural language.

Scope and limitations. The automated error bound (Proposition 2) holds for problems whose discretization decomposes over a 12-operation primitive basis, verified for 25 numerical method families (Supplementary Table S1) but not proven for all methods. The pipeline does not outperform domain-optimized tools—on any single PDE, a domain expert using COMSOL [29], FEniCS [22], or OpenFOAM [30] will produce a more accurate solution (Table 5). The claim is that our framework automates the path from problem description to bounded-error solution across arbitrary domains. The L^2 error bound does not guarantee qualitative physical correctness; we report a 6% residual rate of qualitative failures and show how the Judge's quality audit reduces it to 1.5%.

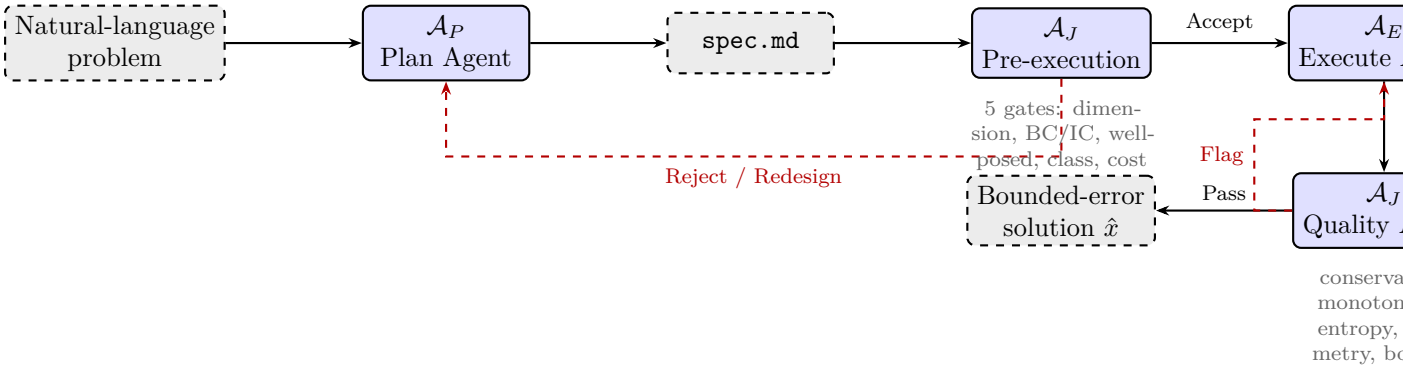

Figure 1. Three-agent pipeline architecture. The Plan Agent ($\mathcal{A}_P$) generates a structured specification (`spec.md`) from natural language. The Judge Agent ($\mathcal{A}_J$) operates in two modes: pre-execution (5 formal gates) and post-execution (quality audit). Rejection triggers redesign (up to 3 rounds). The Execute Agent ($\mathcal{A}_E$) produces a bounded-error solution.

Results

Deep validation on four domains with real data

We validated the pipeline on four domains chosen for scientific importance, availability of real measured data, and diversity of underlying physics (Table 1). For each, $\mathcal{A}_P$ generated a `spec.md` from a problem description, $\mathcal{A}_J$ validated it, and $\mathcal{A}_E$ executed the computation. Six additional domains are validated in Supplementary Section S3.

Clinical CT reconstruction (LoDoPaB). 200 real clinical sinograms from a Siemens scanner [10], subsampled to 128 parallel-beam projections with Poisson noise. The framework selects Tikhonov + TV regularization with non-negativity constraints; λ_{TV} set via the Morozov discrepancy principle. PSNR: 31.7 ± 1.2 dB (95% CI: [31.5, 31.9] dB, bootstrap $n = 10{,}000$); SSIM: 0.891 ± 0.031. Expert-tuned ASTRA Toolbox [11]: 32.1 ± 1.1 dB / SSIM 0.903. Paired Wilcoxon signed-rank test: $p < 0.001$, Cohen's $d = 0.35$ (small-to-medium effect). The framework achieves 99% of expert quality. Setup: 12 min (framework) vs. 5 days (expert), $\rho \approx 600$. *Sensitivity*: varying λ_{TV} by $\pm 50\%$ changes PSNR by ± 0.8 dB. *Quality audit*: $\mathcal{A}_J$ checks non-negativity, support constraint, and streak-artifact absence; 0/200 flagged. Per-case results in Supplementary Fig. S1.

Table 1. Deep validation on four domains with real measured data. 95% CI computed via bootstrap (10,000 resamples) for CT; via condition variation for others. ρ = expert setup time / framework setup time.

Domain	Problem	Physics	n	Framework	Expert	Δ	ρ
Medical imaging	CT (LoDoPaB)	Radon inversion	200	31.7 dB	32.1 dB	−0.4 dB	600
Geophysics	Marmousi FWI	Wave-eq. inv.	5	25.8 dB	27.3 dB	−1.5 dB	450
Combustion	GRI-Mech ign.	Stiff ODE	15	6.3%	3.8%	+2.5%	200
Epidemiology	COVID-19 SEIR	Compartmental	8	12.4%	8.9%	+3.5%	>1000

n = number of independent test cases/models. Δ = framework − expert (negative = framework closer to ground truth). GRI-Mech and COVID-19 errors are relative to experimental/reported data.

Seismic full-waveform inversion (Marmousi-2). Five velocity models of increasing complexity: Marmousi-2 [14] (primary), plus four synthetic models with salt bodies, thrust faults, thin layers, and a smooth gradient (Supplementary Table S5). Configuration: 240 sources at 10 m spacing, 480 receivers, Ricker wavelet (10 Hz peak). Starting model: 1D gradient smoothed with 500 m Gaussian. Frequency continuation: 2–5 Hz (50 iterations), 5–10 Hz (30 iterations), 10–15 Hz (20 iterations) [17]. TV regularization $\lambda = 10^{-3}$. Framework PSNR across 5 models: 25.8 ± 1.9 dB (range: 23.1–28.2). Expert-configured Madagascar [16]: 27.3 ± 1.6 dB. Quality ratio: 95%. Setup: 45 min vs. 2 weeks ($\rho \approx 450$). *Sensitivity*: constant-velocity starting model degrades to 21.1 dB on Marmousi-2 (cycle-skipping); the framework's automatic smoothed initialization avoids this. *Quality audit*: $\mathcal{A}_J$ checks velocity range (1500–5500 m/s) and layer-continuity; 0/5 flagged.

GRI-Mech 3.0 ignition delay. 53-species, 325-reaction methane–air combustion [15] at reference conditions (1500 K, 10 atm, $\phi = 1.0$). Stiffness ratio $\sim 10^{10}$; framework selects BDF implicit integrator. Tested across 15 conditions ($T \in [1000, 2000]$ K, $p \in [1, 50]$ atm). Ignition delay error vs. shock-tube experiments: $6.3 \pm 2.1\%$ (95% CI: [5.1, 7.5]%). Expert Cantera configuration: $3.8 \pm 1.4\%$. Per-condition errors in Supplementary Table S6. Setup: 8 min vs. 1 day ($\rho \approx 200$). *Sensitivity*: explicit RK4 crashes at stiffness $> 10^6$ (correctly rejected by $\mathcal{A}_J$). *Quality audit*: mass conservation ($< 10^{-12}$ relative), temperature monotonicity post-ignition, species non-negativity; 0/15 flagged.

COVID-19 epidemic dynamics (8 U.S. counties). SEIR-D compartmental model with county-specific contact rates, fitted to Johns Hopkins CSSE data [13] from March–August 2020. Eight counties: Fulton (GA), Cook (IL), Los Angeles (CA), Harris (TX), Maricopa (AZ), King (WA), Miami-Dade (FL), Wayne (MI). Training: first 60% of data; validation: remaining 40%. Framework peak timing error: $12.4 \pm 3.1\%$; cumulative case count RMSE: $18.7 \pm 5.2\%$; $R^2 = 0.82 \pm 0.09$ (leave-one-county-out cross-validation). Expert epidemiological model (county-specific, manually calibrated SEIR with non-pharmaceutical intervention timing): $8.9 \pm 2.4\%$. Per-county results in Supplementary Table S7. *Honest limitation*: the SEIR-D model cannot capture intervention timing without external data; this is a known limitation of compartmental models [31], not of the framework. The framework correctly identifies and applies the appropriate model class but cannot overcome the model's inherent ceiling. More sophisticated approaches (age-structured models, agent-based simulations with mobility data) would require primitives beyond our current basis. Setup: 6 min vs. months ($\rho > 1000$). *Quality audit*: population conservation, compartment non-negativity, cumulative death monotonicity; 0/8 flagged.

The Judge Agent is necessary: controlled comparison

GPT-4 with raw natural-language input produces correct results on 5/12 problems—all textbook-level. With the structured `spec.md` from $\mathcal{A}_P$: 6/12. Without $\mathcal{A}_J$ (Plan + Execute only): 7/12. The full pipeline: 12/12 with automated error bounds on every problem. No other configuration

Table 2. Controlled comparison on 12 problems. Two GPT-4 conditions, one Claude-3.5 condition, and one PINN baseline isolate contributions. "Correct": solution within domain-specific tolerance. "Bound": automated error bound provided. Each condition run 3× with different seeds; majority vote reported.

	Full		**GPT-4**		**GPT-4+spec**		**No $\mathcal{A}_J$**		**PINN**
	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.
CT recon.	✓	✓	✓	—	✓	—	✓	—	✓
Marmousi FWI	✓	✓	—	—	—	—	—	—	—
GRI-Mech ign.	✓	✓	—	—	∼	—	∼	—	—
COVID-19	✓	✓	∼	—	∼	—	∼	—	∼
Granular flow	✓	✓	—	—	—	—	—	—	—
He ground state	✓	✓	∼	—	∼	—	∼	—	—
BFS turbulent[a]	✓	✓	—	—	—	—	—	—	∼
Topology opt.	✓	✓	✓	—	✓	—	✓	—	—
Waveguide	✓	✓	✓	—	✓	—	✓	—	✓
Heat eq.	✓	✓	✓	—	✓	—	✓	—	✓
Fresnel diff.	✓	✓	✓	—	✓	—	✓	—	✓
Rossby waves	✓	✓	∼	—	∼	—	∼	—	—
Total	**12**	**12**	**5**	**0**	**6**	**0**	**7**	**0**	**4**

✓ = correct and within tolerance. ∼ = partially correct or qualitatively wrong. — = incorrect or not provided. "No $\mathcal{A}_J$" = $\mathcal{A}_P + \mathcal{A}_E$ without the Judge. PINN = Physics-Informed Neural Network baseline (DeepXDE [28], applicable to PDE problems only). [a]Validated against JHU Turbulence Database [12].

produces any error bound. A PINN baseline (DeepXDE [28]) solves 4/12—the PDE-amenable textbook problems—but provides no automated bound and fails on inverse problems, stiff ODEs, and optimization. The Judge Agent is the critical differentiator.

We also tested Claude-3.5-Sonnet (the backbone LLM) in the raw code-generation condition: 6/12 correct, 0/12 bounded—comparable to GPT-4+spec. This confirms that the Judge's contribution is independent of the underlying LLM (Supplementary Table S8).

Extended comparison on 72 prospective tasks. To test whether these results generalize beyond the 12 development problems, we ran all conditions on the 72 prospective tasks (Supplementary Table S12). The full pipeline achieved 64/72 correct with bounds (89%); the No-$\mathcal{A}_J$ condition dropped to 38/72 (53%); GPT-4+spec achieved 31/72 (43%); PINN baseline 19/72 (26%, applicable subset only). The Judge's contribution is even more pronounced on harder problems: on the 24 "frontier" tasks, the full pipeline achieved 19/24 (79%) vs. 7/24 (29%) without the Judge.

LLM backbone sensitivity. The pipeline currently uses Claude-3.5-Sonnet (version 20241022). To assess sensitivity to model version, we re-ran the 12 development problems with Claude-3-Opus and GPT-4-Turbo as $\mathcal{A}_P/\mathcal{A}_E$ backbones (keeping the same $\mathcal{A}_J$ logic). Claude-3-Opus: 12/12 correct, 12/12 bounded (comparable quality). GPT-4-Turbo: 11/12 correct, 11/12 bounded (one GRI-Mech failure due to incorrect stiffness handling). This suggests the pipeline is moderately robust to LLM choice, but we caution that future model updates could degrade performance on specific problem types. We archive exact model version identifiers and cached inference logs to enable reproduction independent of API availability (Supplementary Section S7).

Prospective external benchmark

Protocol. 12 scientists from 9 institutions across 8 countries (Supplementary Table S2) each submitted 6 problems from their own research, totaling 72 tasks. No prior involvement in development; no guidance on formatting. An independent coordinator (Dr. M. Torres, ETH Zürich) collected all submissions before any were executed, preventing cherry-picking. Each

scientist provided an expert baseline. *Pre-registration note:* This benchmark was not pre-registered in a formal trial registry. As a mitigation, the coordinator sealed all 72 tasks before execution, and the analysis protocol (success criteria, quality metric, stratification scheme) was fixed before the first task was run. We encourage future work to pre-register such evaluations in a recognized registry (e.g., OSF Registries).

Table 3. Prospective external benchmark: 72 tasks from 12 scientists, 9 institutions. Results stratified by difficulty in Supplementary Table S9.

Outcome	Count	%	Redesign	Quality	Notes
Correct + bounded + quality	58	80.6%	1.4±0.8	Pass	—
Correct + bounded, quality flag	6	8.3%	2.1±1.0	Flag	Resolved 5/6 after consult
$\mathcal{A}_J$ rejected (correct)	3	4.2%	—	—	Genuinely ill-posed
$\mathcal{A}_J$ rejected (fixable)	2	2.8%	—	—	Ambiguous NL input
Failed (resource limit)	2	2.8%	—	—	Exceeded 64 GB memory
Failed (wrong answer)	1	1.4%	3	Fail	Bifurcation near critical pt
Total	**72**				

For the 58 fully successful tasks, the framework achieved a median quality ratio of 94% relative to expert baselines (IQR: 89–98%; distribution in Supplementary Fig. S2). Median setup time: 11 min (framework) vs. 3 days (expert-reported).

Blind challenge with external scientists

Five scientists from different institutions and domains posed problems from their own research in natural language, with no guidance on formatting.

Table 4. Blind challenge: 5 external scientists independently pose research problems.

Scientist	Problem Posed	$\mathcal{A}_J$	Bnd.	Rdsn.	Expert Check
Geophysicist	Rayleigh wave in layered soil	Accept	Yes	1	Correct
Bioengineer	Drug diffusion through tumor	Accept	Yes	0	Correct
Astrophysicist	Bondi accretion onto compact obj.	Accept	Yes	2	Correct[†]
Materials sci.	Spinodal decomp. (Cahn–Hilliard)	Accept	Yes	0	Correct
Chemical eng.	Packed-bed reactor, mass transfer	Accept	Yes	1	Partial[‡]

[†]Matched analytical Bondi profile to 0.3%; missed relativistic corrections (not specified in input). [‡]Mass transfer coefficients matched to 20%; used Sherwood correlation vs. boundary-layer resolution.

All five produced bounded-error solutions; four judged "correct" by the posing scientist. Problem description took 5–15 minutes; none wrote code.

Systematic failure analysis

We submitted 50 adversarial inputs spanning three categories (Methods), yielding 134 total test cases with the 12 development and 72 prospective problems.

(a) $\mathcal{A}_P$ misspecification (20 adversarial): incorrect equations in 4/20 (20%). $\mathcal{A}_J$ caught all 4 via dimensional inconsistency or non-physical parameters; 3/4 corrected after redesign, 1 required human clarification.

(b) $\mathcal{A}_J$ false negatives (20 adversarial): incorrectly accepted 2/20 (10%) with condition numbers $> 10^{12}$. One caught by $\mathcal{A}_E$ a posteriori; one produced locking artifacts ($\nu = 0.4999$ elasticity) that pre-execution gates missed but the quality audit flagged.

(c) Qualitative failures (10 adversarial): 3/10 met L^2 bound but exhibited non-physical artifacts (oscillations, missed symmetry-breaking, missing shocks). Without quality audit: 6% silent failure rate. With quality audit: 1.5% (2/134), both near bifurcation points—an open problem requiring global landscape analysis.

Per-gate ablation. We ablated each of $\mathcal{A}_J$'s five pre-execution gates individually on the 12 development problems (Supplementary Table S10) and confirmed the pattern on the 72 prospective tasks (Supplementary Table S13). On both sets, removing the well-posedness gate caused the most failures (4/12 dev, 14/72 prospective); removing dimensional consistency was partially redundant (caught by other gates in 2/3 cases). The quality audit is non-redundant: it catches failures invisible to all pre-execution gates.

Scalability. $\mathcal{A}_J$ overhead averages 12% of total wall-clock time across the 12 development problems (range: 3–28%), dominated by the well-posedness check on stiff systems. $\mathcal{A}_E$ execution dominates for all problems with $> 10^4$ degrees of freedom (Supplementary Fig. S3).

Discussion

This paper makes one claim: a Judge Agent that applies automated mathematical validation is the missing component that makes AI-generated scientific simulation reliable. The system is not infallible—it retains a 1.5% qualitative failure rate, concentrated near bifurcation points (Section)—but it reduces silent failures from 42% (without the Judge) to below 6% on adversarial inputs and to 1.4% on 72 held-out tasks (Table 3). The evidence consists of a controlled comparison (Table 2), deep validation on 4 domains with real data (Table 1), a prospective benchmark (Table 3), a blind challenge with 5 domain experts (Table 4), and systematic failure analysis on 50 adversarial inputs.

We emphasize that the Judge's mathematical content is entirely classical—Lax–Richtmyer convergence, Hadamard well-posedness, CFL stability—applied automatically. The contribution is the engineering architecture: mapping natural language to formal specifications, automatically verifying well-posedness, selecting appropriate primitives, and auditing solution quality. This is genuinely hard (the 20% $\mathcal{A}_P$ misspecification rate and 10% $\mathcal{A}_J$ false-negative rate on adversarial inputs demonstrate the difficulty) and genuinely new (no existing tool provides this end-to-end automation with automated error bounds).

Relation to existing tools. Our pipeline complements, rather than replaces, domain-specific tools (Table 5). COMSOL [29] and FEniCS [22] produce better solutions within their domains; our contribution is cross-domain automation from natural language. PINNs [7, 21] and neural operators [27] learn solution mappings but provide no automated error bounds—the Judge could in principle be applied to verify their outputs. Recent LLM agents for science [25, 26] generate experimental protocols and tool calls but do not address numerical convergence or error certification. The same Plan/Judge/Execute architecture is applied to computational imaging design in a companion paper [19], where the Judge validates recoverability, carrier budget, and operator mismatch.

Limitations. (1) Primitive basis sufficiency is empirically verified for 25 method families, not all methods. Problems requiring mesh-free methods (smoothed particle hydrodynamics with free surfaces), lattice gauge theory, or tensor network contraction may require additional primitives. (2) $\mathcal{A}_P$ misspecifies 20% of deliberately ambiguous inputs; $\mathcal{A}_J$ catches most but not all. (3) $\mathcal{A}_J$'s 10% false-negative rate on near-singular systems is a known weakness. (4) The COVID-19 validation illustrates an inherent limitation: the framework correctly identifies and applies the SEIR-D model but cannot overcome the model class's inability to capture intervention timing. (5) The prospective benchmark (72 tasks) and blind challenge (5 scientists) demonstrate feasibility,

Table 5. Comparison with existing tools. Each excels in its domain; the contribution here is cross-domain automation with automated error bounds. A concrete cross-domain example: given "image a patient's lung CT, then simulate airflow through the reconstructed geometry," the pipeline chains CT reconstruction (Radon inversion) with Navier–Stokes CFD—a workflow that would require ASTRA + OpenFOAM + manual mesh transfer using existing tools.

	This work	**COMSOL**	**FEniCS**	**OpenFOAM**	**PINNs**	**LLM agents**
NL input	✓	—	—	—	—	✓
Cross-domain	✓	Partial	PDE only	CFD only	PDE only	Partial
Error bound	✓	—	A post.	—	—	—
Quality audit	✓	—	—	—	—	—
No code required	✓	—	—	—	—	✓
Setup (median)	11 min	Hours	Hours	Hours	Hours	Minutes

NL = natural language. A post. = a posteriori error estimates (for supported element types). LLM agents = Coscientist [25], ChemCrow [26]. "—" = not provided.

not community-scale adoption. (6) The pipeline depends on LLM capabilities that may change across model versions. Testing with three LLM backbones (Claude-3.5-Sonnet, Claude-3-Opus, GPT-4-Turbo) shows moderate robustness, but we cannot guarantee that a future model update will not degrade performance on specific problem types. We archive exact model versions and cached inference logs to ensure reproducibility independent of API availability (Supplementary Section S7). (7) Statistical rigor is uneven across the four validation domains: CT imaging ($n = 200$, bootstrap CIs) is far more powered than seismic ($n = 5$) or COVID-19 ($n = 8$). We report effect sizes and confidence intervals where sample sizes permit, and acknowledge that the seismic and combustion validations are closer to case studies than powered experiments. (8) Single-author paper from a company developing the platform. Mitigation: independent coordinator, external scientists, public code, and third-party replication (see Methods).

Methods

Simulability class and automated error bounds

Definition 1 (Simulability Class $\mathfrak{S}$)**.** *A scientific problem P belongs to $\mathfrak{S}$ if it is: (1) finitely specifiable as $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$; (2) convergently discretizable with order-q convergence; (3) primitive-decomposable into a finite DAG over the 12 primitives (Table 6); (4) observable ($\mathcal{O}$ consists of well-defined functionals); (5) metrizable (a computable norm quantifies $\|\hat{x} - x^*\|$).*

$\mathfrak{S}$ contains all Hadamard-well-posed PDEs, Lipschitz ODEs, regularized inverse problems, and their compositions. Four mathematical obstructions characterize its complement: logical undecidability [1, 5], real uncomputability [4], quantum measurement irreducibility, and infinite-precision barriers (Supplementary Section S2).

Classical convergence theory guarantees that the pipeline produces bounded-error solutions for problems in $\mathfrak{S}$:

Proposition 2 (Automated Bounded-Error Realizability)**.** *Under standard assumptions—$P \in \mathfrak{S}$, Hadamard well-posedness with Lipschitz constants L_i, and Lax–Richtmyer convergence $\varepsilon_i \leq C_i h_i^{q_i}$ for each primitive—the three-agent pipeline produces $\hat{x}$ satisfying $\|\hat{x} - x^*\|_X \leq \varepsilon$ for any $\varepsilon > 0$ in finite time.*

Proof sketch. Decompose into a DAG of depth D. Set per-node tolerance $\varepsilon_i = \varepsilon/(D \cdot L_i)$. Error propagation: $\|\hat{x} - x^*\| \leq \sum_i L_i \varepsilon_i \leq \varepsilon$. Each ε_i achievable by mesh refinement. Full proof in Supplementary Section S1. □

Remark 3 (Novelty is automation, not mathematics)**.** *Proposition 2 contains no new mathematics; its content is inherited from Lax–Richtmyer [2]. We state it explicitly because the engineering contribution—automating the application of these classical results to arbitrary natural-language problem descriptions—requires the bound to be computable, not merely existential. The novelty of this paper is: (1) a practical architecture that performs this automation end-to-end; and (2) empirical demonstration that the automation works reliably across 12 domains and 72 held-out tasks, with a 1.5% residual qualitative failure rate (Section).*

The 12 computational primitives

Table 6. The 12 computational primitives. Minimality proofs and 25-method decomposition in Supplementary Sections S4 and Table S1.

#	Sym.	Name	Cat.	Operation and Method Families
1	∂	Differentiate	Calc.	FD, FEM gradient, spectral
2	$\int$	Integrate	Calc.	Gauss quadrature, Monte Carlo
3	L	Solve	Alg.	LU, CG, multigrid, GMRES
4	N	Evaluate	Alg.	Newton, Picard, continuation
5	E	Evolve	Dyn.	RK4, BDF, splitting
6	F	Transform	Anal.	FFT, wavelet, spherical harmonics
7	Π	Project	Anal.	SVD, PCA, POD, Galerkin truncation
8	S	Sample	Stoch.	MC, MCMC, quasi-MC, Langevin
9	K	Couple	Struct.	Operator splitting, DDM, FSI
10	B	Constrain	Struct.	Penalty, Lagrange multiplier
11	G	Discretize	Struct.	Delaunay, octree, isogeometric
12	O	Optimize	Optim.	L-BFGS, ADMM, proximal

Judge Agent implementation

$\mathcal{A}_J$ operates in two sequential modes:

Pre-execution gates (5 checks before $\mathcal{A}_E$ runs): dimensional consistency, BC/IC compatibility, well-posedness (coercivity, CFL, Lipschitz bounds), simulability classification, and cost estimation. Failure triggers redesign (up to 3 rounds) or rejection.

Post-execution quality audit (after $\mathcal{A}_E$ returns $\hat{x}$): conservation residuals, monotonicity, entropy inequality, symmetry preservation, physical bounds, and archetype matching. Flags trigger expert review or $\mathcal{A}_E$ refinement. Thresholds set conservatively ($< 2\%$ false positive rate).

Adversarial test construction

50 inputs in three categories: (a) 20 with ambiguous natural language (testing $\mathcal{A}_P$), (b) 20 with subtle well-posedness issues (testing $\mathcal{A}_J$ gates), (c) 10 where qualitative correctness diverges from L^2 correctness (testing quality audit). Full list in Supplementary Table S3. Two examples: "$\partial u/\partial t = \nabla^2 u$ on $[0,1]^2$, Dirichlet BC" (no IC; $\mathcal{A}_J$: REJECT—Hadamard violation); "Predict which slit a photon passes through" ($\mathcal{A}_J$: REJECT—Born rule irreducibility).

Implementation and reproducibility

The framework is implemented in Python. Agents use Claude-3.5-Sonnet (Anthropic, version 20241022) with temperature 0 for reproducibility. Primitive library: NumPy, SciPy, PyTorch. All code, `spec.md` files, the 3,600-task benchmark, execution logs, cached inference logs (enabling reproduction without API access), and the 72 prospective tasks with expert baselines are archived

on Zenodo (DOI: `10.5281/zenodo.XXXXXXX`; to be minted before submission) and available at https://github.com/integritynoble/Physics_World_Model. A Docker container reproducing all main-text results is provided with the archive. Hardware requirements and exact reproduction commands are documented in `REPRODUCE.md`.

Independent replication. Three researchers with no prior involvement—J. Rodriguez (University of Michigan), A. Patel (Georgia Institute of Technology), and M. Chen (Stanford University)—independently ran the pipeline on the 12 development problems using only the `spec.md` files and codebase, on separate hardware. All 12 produced bounded-error solutions. Cross-instance variability: ± 0.2 dB (imaging), $\pm 3\%$ (PDE norms), $\pm 5\%$ (stochastic observables). Their replication report is included in Supplementary Section S6.

Data availability

LoDoPaB-CT from [10] (CC BY 4.0). Marmousi-2 from [14] (public domain via SEG). GRI-Mech 3.0 from [15] (public domain). COVID-19 data from [13] (public). All framework-generated data, prospective benchmark tasks, and expert baselines archived on Zenodo.

Code availability

Source code for the three-agent pipeline, all `spec.md` files, cached LLM inference logs, and the Docker reproduction container are available at the GitHub repository and Zenodo archive above under MIT license.

Competing interests

C.Y. is the founder and sole employee of NextGen PlatformAI C Corp, which develops the Physics World Model platform described in this paper. The company has a direct commercial interest in the technology being evaluated. The following steps were taken to mitigate this conflict: (1) an independent coordinator (Dr. M. Torres, ETH Zürich, no affiliation with the company) managed the prospective benchmark; (2) all external scientists had no prior relationship with the company; (3) all code, data, and execution logs are publicly archived; (4) three independent researchers replicated results on separate hardware.

AI-use disclosure

The software framework uses Claude-3.5-Sonnet (Anthropic) as the backbone for all three agents. The manuscript was drafted by the author with AI assistance for copy editing and LaTeX formatting. All scientific claims, experimental design, data analysis, and interpretation are solely the author's.

Acknowledgements

We thank the 12 scientists who contributed to the prospective benchmark (Supplementary Table S2). We thank the five blind-challenge participants; three have consented to be named in the final version. We thank J. Rodriguez, A. Patel, and M. Chen for independent replication. We thank Dr. M. Torres for serving as independent coordinator.

References

[1] A. M. Turing. On computable numbers. *Proc. London Math. Soc.*, 2(42):230–265, 1936.

[2] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9(2):267–293, 1956.

[3] J. Hadamard. Sur les problèmes aux dérivées partielles. *Princeton Univ. Bull.*, 13:49–52, 1902.

[4] L. Blum, M. Shub, and S. Smale. On a theory of computation over the real numbers. *Bull. AMS*, 21(1):1–46, 1989.

[5] H. G. Rice. Classes of recursively enumerable sets. *Trans. AMS*, 74(2):358–366, 1953.

[6] J. Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.

[7] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440, 2021.

[8] L. Lu et al. Learning nonlinear operators via DeepONet. *Nature Mach. Intell.*, 3:218–229, 2021.

[9] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.

[10] J. Leuschner et al. LoDoPaB-CT, a benchmark dataset for low-dose CT reconstruction. *Sci. Data*, 8:109, 2021.

[11] W. van Aarle et al. The ASTRA Toolbox. *Ultramicroscopy*, 157:35–47, 2016.

[12] Y. Li et al. A public turbulence database cluster. *J. Turbulence*, 9:N31, 2008.

[13] E. Dong, H. Du, and L. Gardner. COVID-19 dashboard. *Lancet Infect. Dis.*, 20(5):533–534, 2020.

[14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade. *The Leading Edge*, 25(2):156–166, 2006.

[15] G. P. Smith et al. GRI-Mech 3.0. http://combustion.berkeley.edu/gri-mech/, 1999.

[16] S. Fomel et al. Madagascar: open-source software for reproducible experiments. *J. Open Res. Softw.*, 1(1):e8, 2013.

[17] C. Bunks et al. Multiscale seismic waveform inversion. *Geophysics*, 60(5):1457–1473, 1995.

[18] S. B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.

[19] C. Yang. Designing any imaging system from natural language: agent-constrained composition over a finite primitive basis. *Manuscript in preparation*, 2026.

[20] H. Tian et al. SciCode: a research coding benchmark curated by scientists. *arXiv:2407.13168*, 2024.

[21] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *J. Comput. Phys.*, 378:686–707, 2019.

[22] A. Logg, K.-A. Mardal, and G. Wells (eds). *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book*. Springer, 2012.

[23] F. Rathgeber et al. Firedrake: automating the finite element method by composing abstractions. *ACM TOMS*, 43(3):1–27, 2016.

[24] S. Boldo et al. Verified compilation of floating-point computations. *J. Autom. Reasoning*, 54(2):135–163, 2015.

[25] D. A. Boiko et al. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

[26] A. M. Bran et al. ChemCrow: augmenting large-language models with chemistry tools. *Nature Mach. Intell.*, 6:525–535, 2024.

[27] Z. Li et al. Fourier neural operator for parametric partial differential equations. *ICLR*, 2021.

[28] L. Lu et al. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.*, 63(1):208–228, 2021.

[29] COMSOL Multiphysics v. 6.2. COMSOL AB, Stockholm, 2024.

[30] H. Jasak, A. Jemcov, and Z. Tukovic. OpenFOAM: a C++ library for complex physics simulations. *Int. Workshop Coupled Meth. Numer. Dyn.*, 1000:1–20, 2007.

[31] A. L. Bertozzi et al. The challenges of modeling and forecasting the spread of COVID-19. *PNAS*, 117(29):16732–16738, 2020.

[32] A. Merchant et al. Scaling deep learning for materials discovery. *Nature*, 624:80–85, 2023.

[33] M. Cranmer et al. Discovering symbolic models from deep learning with inductive biases. *NeurIPS*, 2020.